

- Description

This module manages a smart liquid level monitoring and auto-refill system, ensuring that:

1. **The liquid level in a storage tank is continuously monitored to maintain a stable supply.**
2. **If the liquid level drops below the minimum threshold, the refill system is activated.**
3. **If the liquid level rises above the maximum threshold, the refill system is deactivated to prevent overflow.**
4. **A log of all liquid level readings, refill activations, and shutdowns is maintained for resource management.**

This system helps optimize liquid storage, prevent shortages, and automate refill operations.

- Specification

module SmartLiquidLevelControlEx

var currentLiquidLevel, maxLiquidLevel, minLiquidLevel: real

var refillActive: boolean

process updateLiquidLevel(changeAmount: real)

functional scenario form:

currentLiquidLevel >= minLiquidLevel && Liquid level maintained.

||

currentLiquidLevel < minLiquidLevel && refillActive = true, Refill system activated.

||

currentLiquidLevel > maxLiquidLevel && refillActive = false, Refill system deactivated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```

import java.util.List;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c130_SmartLiquidSystemEx
 * Domain: Smart liquid tank with refill control and process monitoring
 *
 * Goals:
 * - Preserve the original decision **inside one core function**:
 *   public synchronized void updateLiquidLevel(double changeAmount)
 *   Branches under test:
 *   (1) currentLiquidLevel += changeAmount
 *   (2) if (currentLiquidLevel < minLiquidLevel) refillActive = true
 *       else if (currentLiquidLevel > maxLiquidLevel) refillActive
= false
 *   (3) print/log states (kept)
 *   Other features (sensors, simulation, GUI, alerts) call this same
core function
 *   so you can focus coverage on this method.
 *
 * Features:
 * - Sensors: level, temperature, viscosity, flow-
in/out estimation, quality index
 * - Controller: RefillActuator (physical clamps/rate), NOT making the
on/off decision
 * - Logger + Alerts (file-backed)
 * - Swing GUI: live status, thresholds, operations, simulation, alert
s, log tail
 * - Simulation engine: inflow/outflow/evap/leak + temperature drift
 * - Replay engine: parse "Env step: ..." from log and replay
 * - CLI harness: VT/FT/Z3/UVT with "comment 3 lines to run only one"
 *
 * Compile: javac code/c130_SmartLiquidSystemEx.java
 * Run GUI: java code.c130_SmartLiquidSystemEx
 * Run CLI: java code.c130_SmartLiquidSystemEx --cli
 * =====
 */
/* ===== Utilities ===== */
class LStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").for
mat(v); }

```

```

        static String fmt3(double v) { return new DecimalFormat("0.000").format(v); }
        static String now() { return LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
    }
    /* ===== Logger ===== */
    class LDataLogger {
        private final File file;
        private final ConcurrentLinkedQueue<String> inMemory = new ConcurrentLinkedQueue<>();
        private final int cap;
        LDataLogger(String path, int inMemoryCap) {
            this.file = new File(path);
            this.cap = Math.max(200, inMemoryCap);
            try {
                if (!file.exists()) {
                    File parent = file.getParentFile();
                    if (parent != null) parent.mkdirs();
                    file.createNewFile();
                }
            } catch (IOException e) {
                System.err.println("[Logger] Create file failed: " + e.getMessage());
            }
        }
        synchronized void log(String message) {
            String line = "[" + LStr.now() + "] " + message;
            inMemory.add(line);
            while (inMemory.size() > cap) inMemory.poll();
            try (FileOutputStream fos = new FileOutputStream(file, true);
                OutputStreamWriter osw = new OutputStreamWriter(fos, StandardCharsets.UTF_8);
                BufferedWriter bw = new BufferedWriter(osw)) {
                bw.write(line);
                bw.newLine();
            } catch (IOException e) {
                System.err.println("[Logger] Write failed: " + e.getMessage());
            }
        }
        List<String> recent(int n) {
            ArrayList<String> list = new ArrayList<>(inMemory);
            if (list.size() <= n) return list;
            return list.subList(list.size() - n, list.size());
        }
    }

```

```

    }
    File getFile() { return file; }
}
/* ===== Alerts ===== */
class LAlertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
        Alert(Severity s, String m) { this.severity = s; this.message =
m; }
        @Override public String toString() { return "[" + time + "]" +
severity + " " + message; }
    }
    private final List<Alert> alerts = new ArrayList<>();
    private final LDataLogger logger;
    LAlertSystem(LDataLogger logger) { this.logger = logger; }
    void push(Severity s, String msg) {
        Alert a = new Alert(s, msg);
        alerts.add(a);
        logger.log("ALERT " + s + " :: " + msg);
        System.out.println(a);
    }
    List<Alert> recent(int n) {
        if (alerts.size() <= n) return new ArrayList<>(alerts);
        return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
    }
}
/* ===== Sensors ===== */
abstract class LSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    LSensor(String name) { this.name = name; }
    abstract void update(LiquidContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}

```

```

class LiquidContext {
    public double currentLiquidLevel; // liters
    public double minLiquidLevel;    // liters
    public double maxLiquidLevel;    // liters
    public boolean refillActive;
    public double tankTemperature;   // °C
    public double viscosity;         // mPa·s (toy)
    public double inflowRate;        // L/s
    public double outflowRate;       // L/s
    public double qualityIndex;      // 0~1 (toy)
    public double cumulativeInflow;   // L
    public double cumulativeOutflow;  // L

    LiquidContext copy() {
        LiquidContext c = new LiquidContext();
        c.currentLiquidLevel = currentLiquidLevel;
        c.minLiquidLevel = minLiquidLevel;
        c.maxLiquidLevel = maxLiquidLevel;
        c.refillActive = refillActive;
        c.tankTemperature = tankTemperature;
        c.viscosity = viscosity;
        c.inflowRate = inflowRate;
        c.outflowRate = outflowRate;
        c.qualityIndex = qualityIndex;
        c.cumulativeInflow = cumulativeInflow;
        c.cumulativeOutflow = cumulativeOutflow;
        return c;
    }
}

class LevelSensor extends LSensor {
    LevelSensor() { super("LevelSensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = Math.max(0, ctx.currentLiquidLevel + (rnd.nextDouble()
- 0.5) * 0.3);
        ok = value >= 0 && value <= Math.max(ctx.maxLiquidLevel * 1.5,
1000.0);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class TemperatureSensorL extends LSensor {
    TemperatureSensorL() { super("TemperatureSensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = ctx.tankTemperature + (rnd.nextDouble() - 0.5) * 0.4;
    }
}

```

```

        ok = value > -10 && value < 120;
        status = ok ? (value > 80 ? "HOT" : "OK") : "OUT_OF_RANGE";
    }
}

class ViscositySensor extends LSensor {
    ViscositySensor() { super("ViscositySensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = Math.max(0, ctx.viscosity + (rnd.nextDouble() - 0.5) *
0.5);
        ok = value >= 0 && value < 10000;
        status = ok ? (value > 1000 ? "HIGH" : "OK") : "BROKEN";
    }
}

class FlowInSensor extends LSensor {
    FlowInSensor() { super("FlowInSensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = Math.max(0, ctx.inflowRate + (rnd.nextDouble() - 0.5) *
0.05);
        ok = value >= 0 && value < 100;
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class FlowOutSensor extends LSensor {
    FlowOutSensor() { super("FlowOutSensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = Math.max(0, ctx.outflowRate + (rnd.nextDouble() - 0.5)
* 0.05);
        ok = value >= 0 && value < 100;
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class QualitySensor extends LSensor {
    QualitySensor() { super("QualitySensor"); }
    @Override
    void update(LiquidContext ctx) {
        value = Math.max(0, Math.min(1.0, ctx.qualityIndex + (rnd.nextD
ouble() - 0.5) * 0.02));
        ok = value >= 0 && value <= 1.0;
        status = ok ? (value < 0.4 ? "POOR" : (value < 0.7 ? "FAIR" : "
GOOD")) : "BROKEN";
    }
}

```

```

}
/* ===== Refill Actuator (physical helper) =====
===== */
class RefillActuator {
    private double maxRate = 2.0; // L/s (toy)
    private boolean valveOpen = false;
    void setValve(boolean open) { valveOpen = open; }
    boolean isValveOpen() { return valveOpen; }
    // clamp how much inflow we can add per second due to physical limit
    double clampInflow(double desiredRate) {
        if (!valveOpen) return 0.0;
        return Math.max(0.0, Math.min(maxRate, desiredRate));
    }
    void setMaxRate(double r) { maxRate = Math.max(0.5, Math.min(20.0, r)); }
    double getMaxRate() { return maxRate; }
}
/* ===== Core Domain (SmartLiquidTank) =====
===== */
class SmartLiquidTank {
    private final LDataLogger logger;
    private final LAlertSystem alert;
    // thresholds
    private double maxLiquidLevel; // L
    private double minLiquidLevel; // L
    // state
    private double currentLiquidLevel; // L
    private boolean refillActive; // decision under test
    // environment / process variables
    private double tankTemperature = 25.0; // °C
    private double viscosity = 10.0; // mPa·s
    private double inflowRate = 0.5; // L/s (desired)
    private double outflowRate = 0.4; // L/s (process demand)
    private double qualityIndex = 0.85; // 0~1
    private double cumulativeInflow = 0.0; // L
    private double cumulativeOutflow = 0.0; // L
    // components
    private final RefillActuator actuator = new RefillActuator();
    private final LevelSensor levelSensor = new LevelSensor();
    private final TemperatureSensorL tempSensor = new TemperatureSensorL();
    private final ViscositySensor viscSensor = new ViscositySensor();
    private final FlowInSensor inflowSensor = new FlowInSensor();

```

```

    private final FlowOutSensor outflowSensor = new FlowOutSensor();
    private final QualitySensor qualSensor = new QualitySensor();
    private final List<Runnable> listeners = new ArrayList<>();

    SmartLiquidTank(double maxLiquidLevel, double minLiquidLevel, double initialLiquidLevel,
        LDataLogger logger, LAlertSystem alert) {
        this.maxLiquidLevel = maxLiquidLevel;
        this.minLiquidLevel = minLiquidLevel;
        this.currentLiquidLevel = initialLiquidLevel;
        this.refillActive = false; // initial
        this.logger = logger;
        this.alert = alert;
        logger.log("SmartLiquidTank initialized. Level=" + LStr.fmt2(currentLiquidLevel) + " L"
            + " (min=" + LStr.fmt2(minLiquidLevel) + ", max=" + LStr.fmt2(maxLiquidLevel) + ")");
    }
    void addListener(Runnable r) { listeners.add(r); }
    private void emit() { for (Runnable r : listeners) try { r.run(); } catch (Throwable ignored) {} }
    LiquidContext buildContext() {
        LiquidContext c = new LiquidContext();
        c.currentLiquidLevel = currentLiquidLevel;
        c.minLiquidLevel = minLiquidLevel;
        c.maxLiquidLevel = maxLiquidLevel;
        c.refillActive = refillActive;
        c.tankTemperature = tankTemperature;
        c.viscosity = viscosity;
        c.inflowRate = inflowRate;
        c.outflowRate = outflowRate;
        c.qualityIndex = qualityIndex;
        c.cumulativeInflow = cumulativeInflow;
        c.cumulativeOutflow = cumulativeOutflow;
        return c;
    }
    private void sampleSensors() {
        LiquidContext ctx = buildContext();
        levelSensor.update(ctx);
        tempSensor.update(ctx);
        viscSensor.update(ctx);
        inflowSensor.update(ctx);
        outflowSensor.update(ctx);
        qualSensor.update(ctx);
        // Alerts
    }

```



```

        if (currentLiquidLevel < minLiquidLevel) {
            alert.push(LAlertSystem.Severity.WARNING, "Below MIN level
(" + LStr.fmt2(currentLiquidLevel) + " < " + LStr.fmt2(minLiquidLevel)
+ " L)");
        }
        if (currentLiquidLevel > maxLiquidLevel) {
            alert.push(LAlertSystem.Severity.WARNING, "Above MAX level
(" + LStr.fmt2(currentLiquidLevel) + " > " + LStr.fmt2(maxLiquidLevel)
+ " L)");
        }
        if (tempSensor.getValue() > 75) {
            alert.push(LAlertSystem.Severity.INFO, "High tank temperatu
re: " + LStr.fmt2(tempSensor.getValue()) + "°C");
        }
        if (qualSensor.getValue() < 0.4) {
            alert.push(LAlertSystem.Severity.CRITICAL, "Liquid quality
poor: idx=" + LStr.fmt2(qualSensor.getValue()));
        }
    }
}
/* =====
* CORE FUNCTION: Keep the original logic in ONE function.
* Branches to cover:
*   - currentLiquidLevel += changeAmount;
*   - if (< min) refillActive = true;
*   - else if (> max) refillActive = false;
*   - Print/log statements retained.
* Extended tasks (actuator/physics/sensors) happen around it
* but the decision remains exclusively here.
* ===== */
public synchronized void updateLiquidLevel(double changeAmount) {
    // (1) Update Level by delta from external causes (in/out/evap/
Leak/manual)
    currentLiquidLevel += changeAmount;
    // if (currentLiquidLevel < 0) currentLiquidLevel = 0; // physica
l clamp
    // (2) Original decision logic in one place
    if (currentLiquidLevel < minLiquidLevel) {
        refillActive = true;
    } else if (currentLiquidLevel > maxLiquidLevel) {
        refillActive = false;
    } // between: keep state
    // (3) Actuator usage (does not make the decision):
    actuator.setValve(refillActive);
    double effectiveInflow = actuator.clampInflow(inflowRate);

```

```

        // update cumulative flow per "update" call (assume ~1s step in
sim/CLI)
        cumulativeInflow += effectiveInflow; // L per step (toy)
        cumulativeOutflow += Math.max(0, outflowRate); // L per step (t
oy)

        // (4) Log & console (kept compatible with original)
        String line = "Current Liquid Level: " + LStr.fmt2(currentLiqui
dLevel) + " liters";
        logger.log(line);
        System.out.println(line);
        if (refillActive) System.out.println("Refill System Activated."
);
        else System.out.println("Refill System Deactivated.");
        // (5) Sensors & UI
        sampleSensors();
        emit();
    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // Apply flows for a small physics-like integration
        double effectiveInflow = actuator.clampInflow(inflowRate) * sec
onds; // L
        double effectiveOutflow = Math.max(0, outflowRate) * seconds;
        // L
        double evap = Math.max(0, 0.002 * seconds);
        // evaporation
        double leak = Math.max(0, (viscosity < 5 ? 0.0015 : 0.0005) * s
econds);
        // net change
        double delta = effectiveInflow - effectiveOutflow - evap - leak
;

        // Use the same core function to keep coverage focus there
        updateLiquidLevel(delta);
        // temperature drift (toy)
        tankTemperature += ((25 + (refillActive ? -
2 : +1)) - tankTemperature) * 0.01;
        // viscosity depends on temperature (toy inverse)
        viscosity = Math.max(1.0, 1000.0 / Math.max(1.0, tankTemperatur
e));
    }
    synchronized void applyEnvironment(double inflowDelta, double outfl
owDelta, double tempDelta, double qualityDelta) {
        inflowRate = Math.max(0, inflowRate + inflowDelta);
        outflowRate = Math.max(0, outflowRate + outflowDelta);
    }

```

```

        tankTemperature += tempDelta;
        qualityIndex = Math.max(0, Math.min(1.0, qualityIndex + quality
Delta));
        logger.log(String.format(Locale.US,
            "Env step: dIn=%.3f L/s, dOut=%.3f L/s, dT=%.3f °C, dQ=%
.3f | In=%.2f, Out=%.2f, T=%.2f°C, Q=%.2f",
            inflowDelta, outflowDelta, tempDelta, qualityDelta, inf
lowRate, outflowRate, tankTemperature, qualityIndex));
        emit();
    }
    /* ===== Maintenance/Config ===== */
    synchronized void calibrateLevel(double observedLiters) {
        double old = currentLiquidLevel;
        currentLiquidLevel = Math.max(0, observedLiters);
        logger.log("Calibration: level " + LStr.fmt2(old) + " -> " + LS
tr.fmt2(currentLiquidLevel) + " L");
        sampleSensors();
        emit();
    }
    synchronized void setThresholds(double maxL, double minL) {
        double oldMax = maxLiquidLevel, oldMin = minLiquidLevel;
        maxLiquidLevel = Math.max(minL + 1, maxL);
        minLiquidLevel = Math.min(minL, maxLiquidLevel - 1);
        logger.log("Thresholds changed: max " + LStr.fmt2(oldMax) + "->
" + LStr.fmt2(maxLiquidLevel) +
            ", min " + LStr.fmt2(oldMin) + "->" + LStr.fmt2(minLiqu
idLevel));
        sampleSensors();
        emit();
    }
    synchronized void setActuatorMaxRate(double r) {
        actuator.setMaxRate(r);
        logger.log("Actuator max inflow rate set to " + LStr.fmt2(r) +
" L/s");
    }
    /* ===== Getters for GUI/Tests ===== */
    double getCurrentLiquidLevel() { return currentLiquidLevel; }
    double getMinLiquidLevel() { return minLiquidLevel; }
    double getMaxLiquidLevel() { return maxLiquidLevel; }
    boolean isRefillActive() { return refillActive; }
    double getTankTemperature() { return tankTemperature; }
    double getViscosity() { return viscosity; }
    double getInflowRate() { return inflowRate; }
    double getOutflowRate() { return outflowRate; }

```

```

    double getQualityIndex() { return qualityIndex; }
    double getCumulativeInflow() { return cumulativeInflow; }
    double getCumulativeOutflow() { return cumulativeOutflow; }
    LevelSensor getLevelSensor() { return levelSensor; }
    TemperatureSensorL getTempSensor() { return tempSensor; }
    ViscositySensor getViscSensor() { return viscSensor; }
    FlowInSensor getFlowInSensor() { return inflowSensor; }
    FlowOutSensor getFlowOutSensor() { return outflowSensor; }
    QualitySensor getQualitySensor() { return qualSensor; }
}
/* ===== Simulation Engine =====
= */
class LiquidSimulationEngine implements Runnable {
    private final SmartLiquidTank tank;
    private final LDataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    private volatile double dIn = +0.02; // inflow drift per tick (L/s
)
    private volatile double dOut = +0.01; // outflow drift per tick (L/
s)
    private volatile double dT = +0.05; // temperature drift per tick
(°C)
    private volatile double dQ = -0.002; // quality drift per tick
    private volatile double tickSeconds = 1.0;
    private volatile long sleepMillis = 250L;
    private volatile boolean randomize = true;
    private final Random rnd = new Random();
    LiquidSimulationEngine(SmartLiquidTank tank, LDataLogger logger) {
        this.tank = tank;
        this.logger = logger;
    }
    void start() {
        if (running.get()) return;
        running.set(true);
        thread = new Thread(this, "LiquidSimEngine");
        thread.setDaemon(true);
        thread.start();
        logger.log("Liquid simulation started.");
    }
    void stop() {
        running.set(false);
        if (thread != null) {

```

```

        try { thread.join(2000); } catch (InterruptedException ignored) {}
    }
    logger.log("Liquid simulation stopped.");
}
boolean isRunning() { return running.get(); }
void setParams(double dIn, double dOut, double dT, double dQ,
               double tickSeconds, long delayMs, boolean randomize)
{
    this.dIn = dIn;
    this.dOut = dOut;
    this.dT = dT;
    this.dQ = dQ;
    this.tickSeconds = Math.max(0.1, tickSeconds);
    this.sleepMillis = Math.max(20, delayMs);
    this.randomize = randomize;
    logger.log(String.format(Locale.US,
        "Sim params updated: dIn=%.4f, dOut=%.4f, dT=%.4f, dQ=%.4f, dt=%.2fs, delay=%dms, random=%s",
        dIn, dOut, dT, dQ, this.tickSeconds, this.sleepMillis,
        String.valueOf(this.randomize)));
}
@Override
public void run() {
    while (running.get()) {
        double di = dIn, dof = dOut, dt = dT, dq = dQ;
        if (randomize) {
            di += (rnd.nextDouble() - 0.5) * Math.max(0.05, Math.abs(dIn) * 2.0);
            dof += (rnd.nextDouble() - 0.5) * Math.max(0.05, Math.abs(dOut) * 2.0);
            dt += (rnd.nextDouble() - 0.5) * Math.max(0.2, Math.abs(dT) * 2.0);
            dq += (rnd.nextDouble() - 0.5) * Math.max(0.01, Math.abs(dQ) * 2.0);
        }
        tank.applyEnvironment(di, dof, dt, dq);
        tank.tickSeconds(tickSeconds);
        try { Thread.sleep(sleepMillis); }
        catch (InterruptedException e) { break; }
    }
}
}
/* ===== RePlay Engine ===== */

```

```

class LiquidReplayEngine {
    private final SmartLiquidTank tank;
    private final LDataLogger logger;
    LiquidReplayEngine(SmartLiquidTank tank, LDataLogger logger) {
        this.tank = tank;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
        logger.log("Replay started from: " + logFile.getAbsolutePath());
;
        try (BufferedReader br = new BufferedReader(new InputStreamReader(
new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                try {
                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    // Env step: dIn=..., dOut=..., dT=..., dQ=... | ..
                    .
                    String[] parts = seg.split("\\|")[0].split(",");
                    double di = Double.parseDouble(parts[0].split("=")[
1].replace("L/s", "").trim());
                    double dof= Double.parseDouble(parts[1].split("=")[
1].replace("L/s", "").trim());
                    double dt = Double.parseDouble(parts[2].split("=")[
1].replace("°C", "").trim());
                    double dq = Double.parseDouble(parts[3].split("=")[
1].trim());

                    tank.applyEnvironment(di, dof, dt, dq);
                    tank.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) {}
            }
        } catch (IOException e) {
            System.err.println("[Replay] Error: " + e.getMessage());
        }
        logger.log("Replay finished.");
    }
}

```

```

/* ===== GUI ===== */
class LiquidGUI extends JFrame {
    private final SmartLiquidTank tank;
    private final LDataLogger logger;
    private final LAlertSystem alerts;
    private final LiquidSimulationEngine engine;
    private final LiquidReplayEngine replay;
    private JLabel lblLv1, lblMin, lblMax, lblRefill, lblTin, lblVisc,
    lblIn, lblOut, lblQ, lblCin, lblCout;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog, txtAlerts;
    private JSpinner spMin, spMax, spMaxRate;
    private JSpinner spdIn, spdOut, spdT, spdQ, spDt, spDelay;
    private JCheckBox cbRandomize;
    private JButton btnStart, btnStop, btnReplay, btnCalib, btnAdd, btn
    Sub, btnApplyTh, btnApplyAct;
    private javax.swing.Timer uiTimer;
    LiquidGUI(SmartLiquidTank tank, LDataLogger logger, LAlertSystem al
    erts,
                LiquidSimulationEngine engine, LiquidReplayEngine replay)
    {
        super("SmartLiquid - Refill & Process Control");
        this.tank = tank;
        this.logger = logger;
        this.alerts = alerts;
        this.engine = engine;
        this.replay = replay;
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        buildUi();
        setSize(1220, 800);
        setLocationRelativeTo(null);
        uiTimer = new javax.swing.Timer(500, e -> refreshUi());
        uiTimer.start();
        tank.addListener(this::refreshUi);
    }
    private JPanel titled(String title, JComponent inner) {
        JPanel p = new JPanel(new BorderLayout());
        p.setBorder(BorderFactory.createTitledBorder(title));
        p.add(inner, BorderLayout.CENTER);
        return p;
    }
    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(10,10));

```

```

root.setBorder(new EmptyBorder(10,10,10,10));
setContentPane(root);
/* LEFT: Status & Controls */
JPanel left = new JPanel();
left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
JPanel status = new JPanel(new GridLayout(0,2,6,6));
lblLv1 = new JLabel("-");
lblMin = new JLabel("-");
lblMax = new JLabel("-");
lblRefill = new JLabel("-");
lblTin = new JLabel("-");
lblVisc = new JLabel("-");
lblIn = new JLabel("-");
lblOut = new JLabel("-");
lblQ = new JLabel("-");
lblCin = new JLabel("-");
lblCout = new JLabel("-");
status.add(new JLabel("Level (L):"));      status.add(lblLv1);
status.add(new JLabel("Min (L):"));      status.add(lblMin);
status.add(new JLabel("Max (L):"));      status.add(lblMax);
status.add(new JLabel("Refill:"));      status.add(lblRefill);
1);
status.add(new JLabel("Temp (°C):"));      status.add(lblTin);
status.add(new JLabel("Viscosity:"));      status.add(lblVisc);
;
status.add(new JLabel("Inflow (L/s):"));      status.add(lblIn);
status.add(new JLabel("Outflow (L/s):"));      status.add(lblOut);
status.add(new JLabel("Quality:"));      status.add(lblQ);
status.add(new JLabel("CumIn (L):"));      status.add(lblCin);
status.add(new JLabel("CumOut (L):"));      status.add(lblCout);
;
left.add(titled("Live Status", status));
JPanel th = new JPanel(new GridLayout(0,2,6,6));
spMin = new JSpinner(new SpinnerNumberModel(30.0, 0.0, 10000.0,
1.0));
spMax = new JSpinner(new SpinnerNumberModel(100.0, 0.0, 10000.0
, 1.0));
btnApplyTh = new JButton("Apply Thresholds");
btnApplyTh.addActionListener(e -> {
    double mn = ((Number)spMin.getValue()).doubleValue();
    double mx = ((Number)spMax.getValue()).doubleValue();
    tank.setThresholds(mx, mn);
});
th.add(new JLabel("Min level (L)")); th.add(spMin);

```



```

th.add(new JLabel("Max level (L)")); th.add(spMax);
th.add(new JLabel(" ")); th.add(btnApplyTh);
left.add(Box.createVerticalStrut(6));
left.add(titled("Thresholds", th));
JPanel act = new JPanel(new GridLayout(0,2,6,6));
spMaxRate = new JSpinner(new SpinnerNumberModel(2.0, 0.5, 20.0,
0.5));
btnApplyAct = new JButton("Apply Actuator Max Inflow");
btnApplyAct.addActionListener(e -> tank.setActuatorMaxRate(((Number)spMaxRate.getValue()).doubleValue()));
act.add(new JLabel("Actuator max inflow (L/s)")); act.add(spMaxRate);
act.add(new JLabel(" ")); act.add(btnApplyAct);
left.add(Box.createVerticalStrut(6));
left.add(titled("Actuator", act));
JPanel ops = new JPanel(new GridLayout(0,2,6,6));
btnAdd = new JButton("Manual +Level +10L");
btnSub = new JButton("Manual -Level -10L");
btnCalib = new JButton("Calibrate Level");
btnAdd.addActionListener(e -> tank.updateLiquidLevel(+10.0));
btnSub.addActionListener(e -> tank.updateLiquidLevel(-10.0));
btnCalib.addActionListener(e -> {
    String s = JOptionPane.showInputDialog(this, "Observed level (L):", LStr.fmt2(tank.getCurrentLiquidLevel()));
    if (s == null) return;
    try { tank.calibrateLevel(Double.parseDouble(s)); }
    catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
});
ops.add(btnAdd); ops.add(btnSub);
ops.add(btnCalib); ops.add(new JLabel(" "));
left.add(Box.createVerticalStrut(6));
left.add(titled("Operations", ops));
JPanel sim = new JPanel(new GridLayout(0,2,6,6));
spdIn = new JSpinner(new SpinnerNumberModel(+0.02, -2.0, 2.0, 0.01));
spdOut= new JSpinner(new SpinnerNumberModel(+0.01, -2.0, 2.0, 0.01));
spdT = new JSpinner(new SpinnerNumberModel(+0.05, -5.0, 5.0, 0.01));
spdQ = new JSpinner(new SpinnerNumberModel(-0.002, -1.0, 1.0, 0.001));
spdDt = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1));

```

```

        spDelay = new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10
));
        cbRandomize = new JCheckBox("Randomize", true);
        btnStart = new JButton("Start Simulation");
        btnStop = new JButton("Stop Simulation");
        btnReplay = new JButton("Replay from Log");
        btnStart.addActionListener(e -> engine.setParams(
            ((Number)spdIn.getValue()).doubleValue(),
            ((Number)spdOut.getValue()).doubleValue(),
            ((Number)spdT.getValue()).doubleValue(),
            ((Number)spdQ.getValue()).doubleValue(),
            ((Number)spDt.getValue()).doubleValue(),
            ((Number)spDelay.getValue()).longValue(),
            cbRandomize.isSelected()
        ));
        btnStart.addActionListener(e -> engine.start());
        btnStop.addActionListener(e -> engine.stop());
        btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
        sim.add(new JLabel("ΔIn per tick (L/s)")); sim.add(spdIn);
        sim.add(new JLabel("ΔOut per tick (L/s)")); sim.add(spdOut);
        sim.add(new JLabel("ΔTemp per tick (°C)")); sim.add(spdT);
        sim.add(new JLabel("ΔQuality per tick")); sim.add(spdQ);
        sim.add(new JLabel("dt (s)")); sim.add(spDt);
        sim.add(new JLabel("Delay (ms)")); sim.add(spDelay);
        sim.add(new JLabel(" ")); sim.add(cbRandomize);
        sim.add(btnStart); sim.add(btnStop);
        sim.add(new JLabel(" ")); sim.add(btnReplay);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Simulation", sim));
        getContentPane().add(left, BorderLayout.WEST);
        /* CENTER: Sensors + Alerts */
        JPanel center = new JPanel(new BorderLayout(6,6));
        tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
            @Override public boolean isCellEditable(int row, int column
) { return false; }
        };
        tblSensors = new JTable(tblModel);
        JScrollPane spTbl = new JScrollPane(tblSensors);
        center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
        txtAlerts = new JTextArea(8, 60);
        txtAlerts.setEditable(false);

```

```

        center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
        getContentPane().add(center, BorderLayout.CENTER);
        /* RIGHT: Log tail */
        txtLog = new JTextArea();
        txtLog.setEditable(false);
        JScrollPane spLog = new JScrollPane(txtLog);
        getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);
    }
    private void refreshUi() {
        lblLv1.setText(LStr.fmt2(tank.getCurrentLiquidLevel()));
        lblMin.setText(LStr.fmt2(tank.getMinLiquidLevel()));
        lblMax.setText(LStr.fmt2(tank.getMaxLiquidLevel()));
        lblRefill.setText(tank.isRefillActive() ? "ON" : "OFF");
        lblRefill.setForeground(tank.isRefillActive() ? new Color(0,120
,0) : Color.RED);
        lblTin.setText(LStr.fmt2(tank.getTankTemperature()));
        lblVisc.setText(LStr.fmt2(tank.getViscosity()));
        lblIn.setText(LStr.fmt2(tank.getInflowRate()));
        lblOut.setText(LStr.fmt2(tank.getOutflowRate()));
        lblQ.setText(LStr.fmt2(tank.getQualityIndex()));
        lblCin.setText(LStr.fmt2(tank.getCumulativeInflow()));
        lblCout.setText(LStr.fmt2(tank.getCumulativeOutflow()));
        tblModel.setRowCount(0);
        tblModel.addRow(new Object[]{
            tank.getLevelSensor().getName(),
            LStr.fmt2(tank.getLevelSensor().getValue()),
            tank.getLevelSensor().isOk(), tank.getLevelSensor().getSt
atus()
        });
        tblModel.addRow(new Object[]{
            tank.getTempSensor().getName(),
            LStr.fmt2(tank.getTempSensor().getValue()),
            tank.getTempSensor().isOk(), tank.getTempSensor().getSt
atus()
        });
        tblModel.addRow(new Object[]{
            tank.getViscSensor().getName(),
            LStr.fmt2(tank.getViscSensor().getValue()),
            tank.getViscSensor().isOk(), tank.getViscSensor().getSt
atus()
        });
        tblModel.addRow(new Object[]{

```

```

        tank.getFlowInSensor().getName(),
        LStr.fmt2(tank.getFlowInSensor().getValue()),
        tank.getFlowInSensor().isOk(), tank.getFlowInSensor().g
etStatus()
    });
    tblModel.addRow(new Object[]{
        tank.getFlowOutSensor().getName(),
        LStr.fmt2(tank.getFlowOutSensor().getValue()),
        tank.getFlowOutSensor().isOk(), tank.getFlowOutSensor()
.getStatus()
    });
    tblModel.addRow(new Object[]{
        tank.getQualitySensor().getName(),
        LStr.fmt2(tank.getQualitySensor().getValue()),
        tank.getQualitySensor().isOk(), tank.getQualitySensor()
.getStatus()
    });
    // alerts
    if (txtAlerts != null) {
        StringBuilder sb = new StringBuilder();
        for (LAlertSystem.Alert a : alerts.recent(18)) sb.append(a)
.append("\n");
        txtAlerts.setText(sb.toString());
        txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
    }
    // log tail
    if (txtLog != null) {
        List<String> lines = logger.recent(220);
        StringBuilder sb = new StringBuilder();
        for (String s : lines) sb.append(s).append("\n");
        txtLog.setText(sb.toString());
        txtLog.setCaretPosition(txtLog.getDocument().getLength());
    }
}

/* ===== CLI Harness ===== */
class LiquidCLIHarness {
    private final SmartLiquidTank tank;
    LiquidCLIHarness(SmartLiquidTank tank) { this.tank = tank; }
    void runAll() {
        System.out.println("==== CLI Demo (VT) ====");
        vt();
        System.out.println("==== CLI Demo (FT) ====");
    }
}

```

```

        ft();
        System.out.println("==== CLI Demo (Z3 mimic) =====");
        z3();
        System.out.println("==== CLI Demo (UVT) =====");
        uvt();
    }
    // VT: original vector to trigger <min / between / >max branches
    void vt() {
        tank.updateLiquidLevel(-
25.0); // 50 -> 25 (<30) => refillActive=true
        tank.updateLiquidLevel(20.0); // 25 -> 45 (between) => keep sta
te (true)
        tank.updateLiquidLevel(60.0); // 45 -> 105 (>100) => refillActi
ve=false
        tank.updateLiquidLevel(-
10.0); // 105 -> 95 (between) => keep state (false)
    }
    // FT: extreme numeric stress (same as user's FT)
    void ft() {
        tank.updateLiquidLevel(3.1393069415704);
        tank.updateLiquidLevel(2.5633107921334);
        tank.updateLiquidLevel(1.797693134862);
        tank.updateLiquidLevel(4.092711172425);
    }
    // Z3-like: mirror VT for deterministic coverage
    void z3() {
        tank.updateLiquidLevel(-25.0);
        tank.updateLiquidLevel(20.0);
        tank.updateLiquidLevel(60.0);
        tank.updateLiquidLevel(-10.0);
    }
    // UVT: mirror VT again
    void uvt() {
        tank.updateLiquidLevel(-25.0);
        tank.updateLiquidLevel(20.0);
        tank.updateLiquidLevel(60.0);
        tank.updateLiquidLevel(-10.0);
    }
}
/* ===== Main ===== */
public class c130_SmartLiquidSystemEx {
    public static void main(String[] args) {
        //         boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;

```

```

        // Logger & Alerts
        LDataLogger logger = new LDataLogger("liquid_log.txt", 900);
        LAlertSystem alertSystem = new LAlertSystem(logger);
        // Core tank with user's parameters
        SmartLiquidTank tank = new SmartLiquidTank(100.0, 30.0, 50.0, logger, alertSystem);
        // Engines
        LiquidSimulationEngine engine = new LiquidSimulationEngine(tank, logger);
        LiquidReplayEngine replay = new LiquidReplayEngine(tank, logger);

        // ===== CLI path (sequential 4 groups; comment 3 lines to run single group) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            LiquidCLIHarness harness = new LiquidCLIHarness(tank);
            harness.vt(); // ← keep to run VT
            // harness.ft(); // ← comment out to skip FT
            // harness.z3(); // ← comment out to skip Z3
            // harness.uvt(); // ← comment out to skip UVT
            // extra ticks to exercise actuator & sensors
            for (int i = 0; i < 20; i++) {
                tank.applyEnvironment(+0.02, +0.01, +0.05, -0.002);
                tank.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {}
            }
            logger.log("CLI demo finished.");
            System.out.println("CLI demo finished. Log written to: " + logger.getFile().getAbsolutePath());
            return;
        }

        // ===== GUI path =====
        logger.log("Running in GUI mode.");
        SwingUtilities.invokeLater(() -> {
            LiquidGUI gui = new LiquidGUI(tank, logger, alertSystem, engine, replay);
            gui.setVisible(true);
        });
    }

    private static boolean headless() {
        try { return GraphicsEnvironment.isHeadless(); }
        catch (Throwable t) { return true; }
    }

```

```

    }
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c130_SmartLiquidSystemEx.java	39.3 %	44	68	112
> LAlertSystem	0.0 %	0	2	2
> LiquidReplayEngine	0.0 %	0	8	8
> LiquidSimulationEngine	0.0 %	0	8	8
> LDataLogger	25.0 %	2	6	8
> c130_SmartLiquidSystemEx	50.0 %	3	3	6
> FlowInSensor	50.0 %	3	3	6
> FlowOutSensor	50.0 %	3	3	6
> LevelSensor	50.0 %	3	3	6
> QualitySensor	50.0 %	5	5	10
> TemperatureSensorL	50.0 %	4	4	8
> ViscositySensor	50.0 %	4	4	8
▼ SmartLiquidTank	75.0 %	15	5	20
emit()	50.0 %	1	1	2
tickSeconds(double)	50.0 %	2	2	4
sampleSensors()	75.0 %	6	2	8
updateLiquidLevel(double)	100.0 %	6	0	6
SmartLiquidTank(double, double, do		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0

3. Vibration testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c130_SmartLiquidSystemEx.java	32.8 %	1,093	2,237	3,330
> LiquidContext	5.7 %	3	50	53
> LiquidReplayEngine	6.3 %	9	133	142
> LiquidSimulationEngine	15.4 %	41	225	266
> LiquidCLIHarness	24.2 %	23	72	95
> LStr	54.5 %	12	10	22
> LAlertSystem	55.9 %	33	26	59
> LDataLogger	57.9 %	70	51	121
> RefillActuator	64.1 %	25	14	39
> c130_SmartLiquidSystemEx	68.2 %	75	35	110
> LSensor	69.0 %	20	9	29
▼ SmartLiquidTank	74.2 %	461	160	621
emit()	52.9 %	9	8	17
sampleSensors()	80.4 %	74	18	92
tickSeconds(double)	94.7 %	71	4	75
SmartLiquidTank(double, double, do	100.0 %	92	0	92
applyEnvironment(double, double, d	100.0 %	87	0	87
buildContext()	100.0 %	50	0	50
updateLiquidLevel(double)	100.0 %	78	0	78
> QualitySensor	89.7 %	52	6	58

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c130_SmartLiquidSystemEx.java	33.9 %	38	74	112
> LAlertSystem	0.0 %	0	2	2
> LiquidReplayEngine	0.0 %	0	8	8
> LiquidSimulationEngine	0.0 %	0	8	8
> LDataLogger	25.0 %	2	6	8
> c130_SmartLiquidSystemEx	50.0 %	3	3	6
> FlowInSensor	50.0 %	3	3	6
> FlowOutSensor	50.0 %	3	3	6
> LevelSensor	50.0 %	3	3	6
> QualitySensor	50.0 %	5	5	10
> RefillActuator	50.0 %	1	1	2
▼ SmartLiquidTank	50.0 %	10	10	20
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	4	4	8
tickSeconds(double)	50.0 %	2	2	4
updateLiquidLevel(double)	50.0 %	3	3	6
SmartLiquidTank(double, double, do		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0

5. Fuzz testing instruction coverage: 85.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c130_SmartLiquidSystemEx.java	29.3 %	975	2,355	3,330
> LiquidContext	5.7 %	3	50	53
> LiquidReplayEngine	6.3 %	9	133	142
> LiquidSimulationEngine	15.4 %	41	225	266
> LAlertSystem	18.6 %	11	48	59
> LiquidCLIHarness	24.2 %	23	72	95
> RefillActuator	46.2 %	18	21	39
> LStr	54.5 %	12	10	22
> LDataLogger	57.9 %	70	51	121
> c130_SmartLiquidSystemEx	68.2 %	75	35	110
▼ SmartLiquidTank	68.9 %	428	193	621
emit()	52.9 %	9	8	17
sampleSensors()	56.5 %	52	40	92
updateLiquidLevel(double)	85.9 %	67	11	78
tickSeconds(double)	94.7 %	71	4	75
SmartLiquidTank(double, double, do	100.0 %	92	0	92
applyEnvironment(double, double, d	100.0 %	87	0	87
buildContext()	100.0 %	50	0	50

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c130_SmartLiquidSystemEx.java	39.3 %	44	68	112
> LAlertSystem	0.0 %	0	2	2
> LiquidReplayEngine	0.0 %	0	8	8
> LiquidSimulationEngine	0.0 %	0	8	8
> LDataLogger	25.0 %	2	6	8
> c130_SmartLiquidSystemEx	50.0 %	3	3	6
> FlowInSensor	50.0 %	3	3	6
> FlowOutSensor	50.0 %	3	3	6
> LevelSensor	50.0 %	3	3	6
> QualitySensor	50.0 %	5	5	10
> TemperatureSensorL	50.0 %	4	4	8
> ViscositySensor	50.0 %	4	4	8
▼ SmartLiquidTank	75.0 %	15	5	20
emit()	50.0 %	1	1	2
tickSeconds(double)	50.0 %	2	2	4
sampleSensors()	75.0 %	6	2	8
updateLiquidLevel(double)	100.0 %	6	0	6
SmartLiquidTank(double, double, do		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c130_SmartLiquidSystemEx.java	32.8 %	1,093	2,237	3,330
> LiquidContext	5.7 %	3	50	53
> LiquidReplayEngine	6.3 %	9	133	142
> LiquidSimulationEngine	15.4 %	41	225	266
> LiquidCLIHarness	24.2 %	23	72	95
> LStr	54.5 %	12	10	22
> LAlertSystem	55.9 %	33	26	59
> LDataLogger	57.9 %	70	51	121
> RefillActuator	64.1 %	25	14	39
> c130_SmartLiquidSystemEx	68.2 %	75	35	110
> LSensor	69.0 %	20	9	29
▼ SmartLiquidTank	74.2 %	461	160	621
emit()	52.9 %	9	8	17
sampleSensors()	80.4 %	74	18	92
tickSeconds(double)	94.7 %	71	4	75
SmartLiquidTank(double, double, do	100.0 %	92	0	92
applyEnvironment(double, double, d	100.0 %	87	0	87
buildContext()	100.0 %	50	0	50
updateLiquidLevel(double)	100.0 %	78	0	78
> QualitySensor	89.7 %	52	6	58

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c130_SmartLiquidSystemEx.java	39.3 %	44	68	112
> LAlertSystem	0.0 %	0	2	2
> LiquidReplayEngine	0.0 %	0	8	8
> LiquidSimulationEngine	0.0 %	0	8	8
> LDataLogger	25.0 %	2	6	8
> c130_SmartLiquidSystemEx	50.0 %	3	3	6
> FlowInSensor	50.0 %	3	3	6
> FlowOutSensor	50.0 %	3	3	6
> LevelSensor	50.0 %	3	3	6
> QualitySensor	50.0 %	5	5	10
> TemperatureSensorL	50.0 %	4	4	8
> ViscositySensor	50.0 %	4	4	8
▼ SmartLiquidTank	75.0 %	15	5	20
emit()	50.0 %	1	1	2
tickSeconds(double)	50.0 %	2	2	4
sampleSensors()	75.0 %	6	2	8
updateLiquidLevel(double)	100.0 %	6	0	6
SmartLiquidTank(double, double, do		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0

9. Unimproved V-method instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c130_SmartLiquidSystemEx.java	32.8 %	1,093	2,237	3,330
> LiquidContext	5.7 %	3	50	53
> LiquidReplayEngine	6.3 %	9	133	142
> LiquidSimulationEngine	15.4 %	41	225	266
> LiquidCLIHarness	24.2 %	23	72	95
> LStr	54.5 %	12	10	22
> LAlertSystem	55.9 %	33	26	59
> LDataLogger	57.9 %	70	51	121
> RefillActuator	64.1 %	25	14	39
> c130_SmartLiquidSystemEx	68.2 %	75	35	110
> LSensor	69.0 %	20	9	29
▼ SmartLiquidTank	74.2 %	461	160	621
emit()	52.9 %	9	8	17
sampleSensors()	80.4 %	74	18	92
tickSeconds(double)	94.7 %	71	4	75
SmartLiquidTank(double, double, do	100.0 %	92	0	92
applyEnvironment(double, double, d	100.0 %	87	0	87
buildContext()	100.0 %	50	0	50
updateLiquidLevel(double)	100.0 %	78	0	78
> QualitySensor	89.7 %	52	6	58